

Smart Parking Allocation & Cruising Reduction System Based on Real-Time Reservation Database

1st Haonan Cheng

Dept. of CST

Graduate School of ICT, Ajou University

Suwon, South Korea

haonan.cheng@hotmail.com

2nd Mingyuan Zhou

Dept. of CST

Graduate School of ICT, Ajou University

Suwon, South Korea

ruyezhou020726@gmail.com

Abstract—Urban traffic congestion is significantly exacerbated by vehicles cruising for parking, which wastes fuel, increases emissions, and reduces overall network efficiency. This paper presents a smart parking allocation and cruising reduction system built on SUMO, Python TraCI, and PostgreSQL. The controller keeps reservation state in Python dictionaries, synchronizes parking inventory and run summaries to relational tables, and applies a tiered pricing policy when local occupancy exceeds 70% and 90%. In the database-backed evaluation, Scenario A parks 2,498 of 2,500 planned vehicles by the 7,200 s simulation limit, whereas Scenario B parks all 2,500 vehicles by 3,922 s while also reducing average search time, fuel consumption, and selected emission indicators.

Index Terms—Smart Parking, Cruising for Parking, Microscopic Traffic Simulation, SUMO, Relational Database, Dynamic Pricing, Python TraCI, PostgreSQL.

I. INTRODUCTION

As urbanization accelerates globally, urban mobility faces unprecedented challenges. A pervasive contributor to traffic congestion is “cruising for parking” [1]. In Central Business Districts (CBDs), cruising vehicles can account for 20% to 30% of total traffic volume during peak hours [2].

Traditional parking management relies on static signage or localized IoT sensor networks [3]. While providing real-time status, they lack active system-wide coordination. When multiple vehicles simultaneously target a single available spot, localized bottlenecks emerge, forcing unsuccessful drivers to resume cruising.

To resolve this, modern Intelligent Transportation Systems (ITS) must shift from “reactive searching” to “proactive reservation” [4]. However, simulating and implementing such a system at a microscopic level presents massive computational challenges. This paper bridges the gap between microscopic traffic simulation and enterprise-grade databases. Our core contributions include:

- 1) **Mathematical Formulation:** Modeling parking allocation as a multi-objective optimization problem balancing distance, cost, and network load.
- 2) **High-Performance Reservation State:** Using Python dictionaries inside the TraCI control loop to handle high-frequency reservation queries, with batched PostgreSQL updates to preserve persistent scenario logs without slowing the simulation.

- 3) **Multi-Tier Dynamic Pricing:** Implementing a demand-responsive algorithm that raises base prices by 1.5x and 2.0x at 70% and 90% occupancy, redirecting traffic to peripheral lots organically.
- 4) **Microscopic Evaluation:** Utilizing the TraCI interface to simulate 2,500 planned trips, quantifying completion time, search time, fuel consumption, explicit cruising distance, selected emission metrics, and network speed using the same monitoring pipeline used during execution.

II. RELATED WORK

A. Modeling and Impacts of Cruising

Shoup [1] and Inci [2] documented the economic and environmental ramifications of underpriced parking. However, macroscopic models often fail to capture the complex, agent-level interactions occurring at intersections and parking lot entrances [5].

B. Smart Parking and IoT Integration

Lin et al. [3] surveyed sensor-based parking systems. Geng and Cassandras [4] proposed resource allocation and reservation systems. Yet, many conceptual proposals rely on discrete-event simulators rather than high-fidelity microscopic environments.

C. Dynamic Pricing in Transportation

Dynamic pricing effectively manages peak demand in ride-sharing and tolling. Fosgerau [6] and Pierce [7] analyzed utilization-based parking fee optimization. We embed a multi-tier pricing logic directly into the simulation loop to observe the immediate microscopic impacts on vehicle routing.

D. Data Synchronization in Co-Simulations

Wegener et al. [8] discussed TraCI latency bottlenecks when scaling up to thousands of vehicles. In this project, the TraCI control loop keeps the active reservation state in Python dictionaries and writes durable updates to PostgreSQL at fixed simulation intervals.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

The architecture forms a closed-loop system between the SUMO simulation, the Python TraCI controller, and the PostgreSQL database.

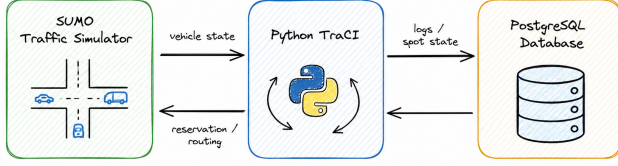


Fig. 1. System architecture showing the data flow between SUMO, Python TraCI, and PostgreSQL.

A. Problem Formulation

The objective is to assign an optimal parking spot $p_j \in P$ to a vehicle $v_i \in V$, minimizing the generalized parking cost. The assignment logic executes as follows:

- 1) **Candidate Spot Filtering (Capacity Constraint):** The system iterates through the reservation-state dictionary, including a spot in the candidate list only if its booked capacity is strictly less than its total capacity:

$$\sum_{i \in V} x_{ij} < \text{Capacity}(p_j), \quad \forall p_j \in P \quad (1)$$

- 2) **Dynamic Price Calculation:** For each available spot, the real-time occupancy rate ρ is retrieved. The current price $\text{Price}(p_j)$ is adjusted based on the 70% and 90% occupancy thresholds.
- 3) **Cost Evaluation (Cost Function):** The generalized cost C_{ij} to park at candidate spot p_j is calculated based on the vehicle's current position:

$$C_{ij} = \text{Price}(p_j) + d_{\text{route}}(v_i, p_j) \cdot c_d \quad (2)$$

Where d_{route} is the estimated route distance from the vehicle to the parking area and c_d is the per-meter monetary distance cost. The implementation sets $c_d = 0.0025$, so 1 km of additional driving is evaluated as 2.5 currency units.

- 4) **Greedy Assignment:** The system extracts the candidate spot that yields the minimum C_{ij} , transforming an expensive global optimization problem into an efficient, low-latency local assignment for each vehicle.

B. Database and State Dictionary Mechanism

The system utilizes a streamlined PostgreSQL schema to minimize transaction overhead while preserving the fields needed for reproducible comparison:

- **parking_spots:** Stores the parking inventory and the dynamic reservation state used by the controller.

- **Static:** spot_id, edge_id, spot_type, capacity, and base_price.
- **Dynamic:** occupied and current_price, synchronized from the Python state dictionary.

- **cruising_logs:** Tracks vehicle-level parking behavior, travel effort, and environmental outputs.

- **Core:** log_id, vehicle_id, scenario, and final_spot_id.
- **Operation:** search_time_sec, cruising_distance_m, and total_fuel_mg.
- **Environment:** total_co2_mg, total_nox_mg, and total_pmx_mg.

- **simulation_runs:** Stores run-level outcomes, including completion_time_sec, total_vehicles, parked_vehicles, failed_vehicles, and parking_rate.

Dictionary-based Reservation Workflow:

- 1) **Initialization:** Before the simulation begins, the parking_spots table is loaded into a Python dictionary.
- 2) **Simulation Loop:** The reservation engine executes $O(1)$ lookups within the dictionary, updating occupancy rates and calculating prices instantly in memory.
- 3) **Batch Synchronization:** Every 60 simulation seconds, the Python TraCI controller writes a bulk UPDATE transaction to PostgreSQL through sync_spots or sync_spots_priced.

C. Multi-Tier Dynamic Pricing Logic

As detailed in Algorithm 1, the current price P_{current} is a piecewise function dependent on the real-time local occupancy rate ρ :

$$P_{\text{current}} = \begin{cases} P_{\text{base}}, & \text{if } \rho \leq 0.70 \\ 1.5 \times P_{\text{base}}, & \text{if } 0.70 < \rho \leq 0.90 \\ 2.0 \times P_{\text{base}}, & \text{if } \rho > 0.90 \end{cases} \quad (3)$$

The implemented Scenario B script evaluates this function at every simulation step before handling newly departed vehicles. Small on-street records are aggregated at street level using STREET_SPOT_THRESHOLD=3; large off-street hubs are priced by their own booked-capacity ratio. Vehicle-to-spot distance is estimated in Python from a cached SUMO network graph using one Dijkstra search per newly departed vehicle, and the cost function uses UNIT_DIST_COST=0.0025 to convert route distance into the same monetary unit as parking price.

D. Cruising Detection and TraCI Integration

In the baseline scenario, vehicles navigate blindly and search from the road network itself. The baseline script builds opposite-edge and outgoing-edge maps from demo.net.xml, scans visible parking areas within a 180 m sight distance, checks intersections every three simulation steps, and reroutes a vehicle when the remaining route is nearly

Algorithm 1 Dynamic Pricing & Reservation Engine

Require: Vehicle v_i , State Dictionary $\mathcal{D}$ **Ensure:** Optimal Parking Spot p_{opt}

```
1:  $min\_cost \leftarrow \infty$ 
2:  $p_{opt} \leftarrow \text{NULL}$ 
3: for each spot  $p_j \in \mathcal{D}$  do
4:   if  $p_j.\text{booked} < p_j.\text{capacity}$  then
5:      $\rho \leftarrow p_j.\text{booked} / p_j.\text{capacity}$ 
6:     if  $\rho > 0.90$  then
7:        $P_{current} \leftarrow p_j.\text{base\_price} \times 2.0$ 
8:     else if  $\rho > 0.70$  then
9:        $P_{current} \leftarrow p_j.\text{base\_price} \times 1.5$ 
10:    else
11:       $P_{current} \leftarrow p_j.\text{base\_price}$ 
12:    end if
13:     $dist \leftarrow \text{LocalGraphRouteDistance}(v_i, p_j)$ 
14:     $cost \leftarrow P_{current} + (dist \times \text{UNIT\_DIST\_COST})$ 
15:    if  $cost < min\_cost$  then
16:       $min\_cost \leftarrow cost$ 
17:       $p_{opt} \leftarrow p_j$ 
18:    end if
19:  end if
20: end for
21: if  $p_{opt} \neq \text{NULL}$  then
22:    $p_{opt}.\text{booked} \leftarrow p_{opt}.\text{booked} + 1$ 
23:   Route vehicle  $v_i$  to  $p_{opt}$  via TraCI
24: end if
25: return  $p_{opt}$ 
```

exhausted. A locked target is abandoned after 120 seconds if the vehicle cannot park there, reproducing the unstable behavior of competitive curbside search.

Scenario B removes this street-scanning loop. Each departed vehicle is assigned once through the in-memory dictionary, routed to the chosen parking area with TraCI, and settled when `traci.vehicle.isStoppedParking` becomes true. The log row for Scenario B therefore records zero explicit cruising distance for the reservation phase; fuel and emission values reflect the normal drive from origin to the assigned parking facility.

IV. EXPERIMENTAL SETUP

The evaluation was conducted using SUMO (version 1.26.0).

A. Network Topology and Capacity

The simulated network is generated by the project script `generate_network.ps1`. It creates a 15×15 grid with 225 non-internal junctions, 840 bidirectional road edges, and 2,520 lanes. Each grid segment is 200 m, each edge has three lanes, the default speed is 11.11 m/s, and SUMO is asked to infer actuated traffic signals.

The parking inventory is generated by `generate_parking.py` and materialized in both `schema.sql` and `parking.add.xml`. The database

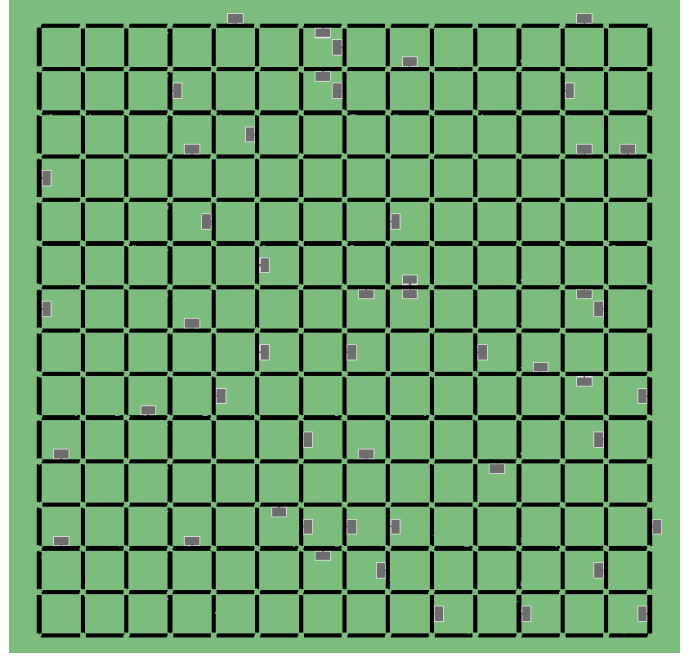


Fig. 2. Overall SUMO road network with generated parking facilities.

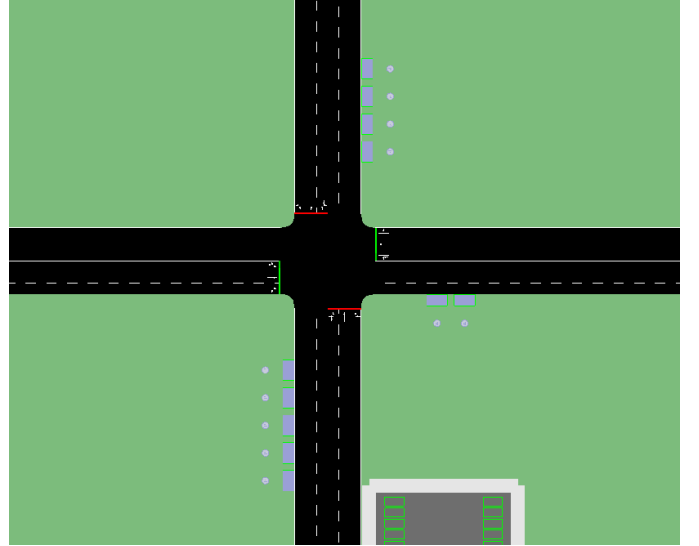


Fig. 3. Intersection-level detail of the generated grid network.

contains 850 parking records, but the physical capacity is 2,700 spaces: 800 one-space on-street records and 50 off-street hubs with 38 spaces each. This distinction matters because the reservation system reasons over database records while SUMO stops vehicles at the corresponding parking-area objects.

Figures 2–4 provide the spatial context used by both scenario scripts: the complete grid, an intersection-level detail view, and an off-street parking facility.

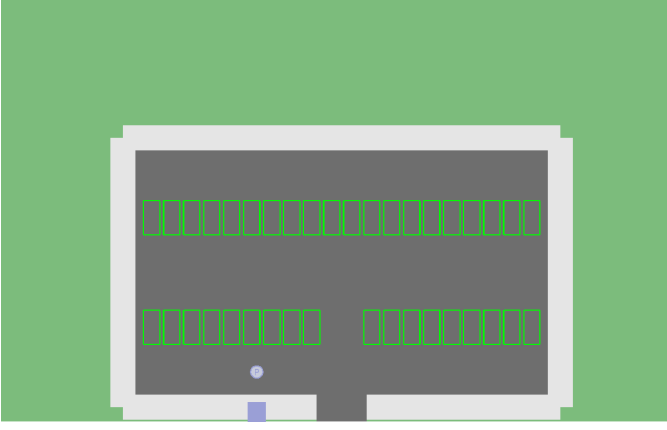


Fig. 4. Off-street parking facility represented in the generated parking inventory.

TABLE I
SIMULATION CONFIGURATION PARAMETERS

Parameter	Configuration Value
Simulator	Eclipse SUMO
Car-Following Model	Krauss [9]
Total Vehicles	2,500
Simulation Duration	7,200 seconds (2 hours)
Trip Departures	3.0–3,599.8 seconds in demo.trips.xml
Parking Records	850 (800 on-street, 50 off-street hubs)
Physical Parking Capacity	2,700 spaces
Database Backend	PostgreSQL 18.4
Control Interface	TraCI via Python 3.10
Pricing Thresholds (ρ)	Tier 1: 70% (1.5x), Tier 2: 90% (2.0x)

B. Simulation Parameters

C. Evaluation Scenarios

- **Scenario A (Baseline Random Search):** The Smart Allocation system is deactivated. Drivers search from the roadway, scan nearby curbside spaces, create pending targets, abandon unavailable spots, and randomly reroute when a route segment is exhausted.
- **Scenario B (Smart Allocation):** The reservation architecture is active. Vehicles query the state dictionary when they depart, receive a unified-cost-minimizing spot, increment the booked count, and synchronize occupancy and price to PostgreSQL in batches.

TABLE II
PROJECT CONSTANTS USED BY THE TWO SCENARIO SCRIPTS

Implementation Setting	Value in Project Code
Baseline sight distance	180 m
Baseline parking scan interval	3 simulation steps
Target waiting timeout	120 seconds
Database sync interval	60 seconds
Smart distance cost	0.0025 currency units per meter
Street aggregation threshold	capacity ≤ 3 spaces
Monitoring refresh interval	5 simulation steps

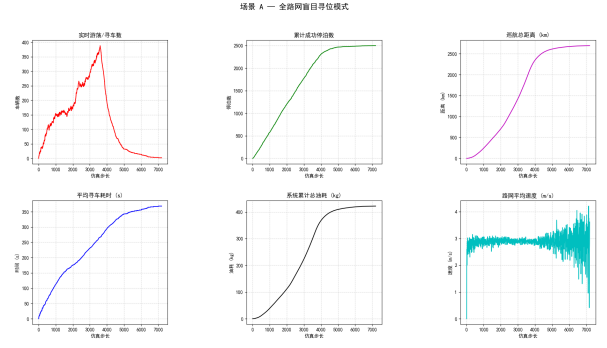


Fig. 5. Scenario A monitoring panel generated by the project Matplotlib monitor: blind network-wide search with active searchers, cumulative parked vehicles, cruising distance, average search time, cumulative fuel, and network speed.

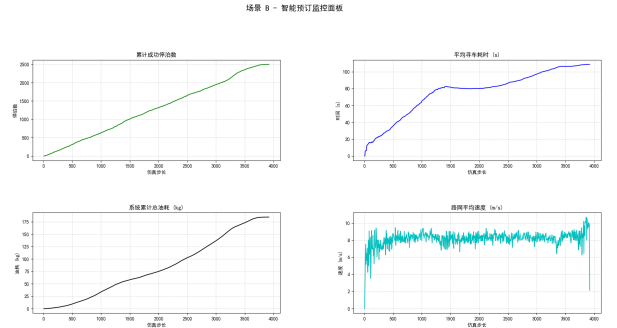


Fig. 6. Scenario B monitoring panel generated by the project Matplotlib monitor: smart reservation with cumulative parked vehicles, average search time, cumulative fuel, and network speed.

V. RESULTS AND PERFORMANCE EVALUATION

Figure 5 shows the baseline monitor. The six-panel layout includes active searching vehicles, cumulative parked vehicles, explicit cruising distance, average parking search time, cumulative fuel, and network average speed. The curve reaches the two-hour boundary rather than a clean all-parked completion: the database log records 2,498 parked vehicles out of 2,500 planned trips, a 99.92% parking rate, with 2,692.06 km of explicit cruising and 371.29 s of average search time.

Figure 6 shows the smart reservation monitor. Its four-panel layout focuses on cumulative successful parking, average search time, cumulative fuel, and network average speed. Scenario B reaches all 2,500 parked vehicles at 3,922 s, maintains a 100.00% parking rate, and records a database average search time of 108.72 s.

The evaluation uses two database sources. The simulation_runs table records the simulation ending time, while Cruising_Logs aggregates vehicle-level parking outcomes, travel effort, fuel, and selected emission indicators including CO₂, NO_x, and PM_x. Parking rate is calculated from vehicle-level parking records, so Scenario A is reported as 2,498 parked vehicles out of 2,500 planned trips. The key operational result remains total simulation

time: Scenario B reaches all-vehicle completion in 3,922 s, whereas Scenario A reaches the two-hour 7,200 s simulation limit with two vehicles unresolved in the vehicle log.

TABLE III
PARKING RATE AND SIMULATION ENDING TIME

Metric	Scenario A	Scenario B
Planned vehicles	2,500	2,500
Parked vehicles	2,498	2,500
Unparked vehicles	2	0
Parking rate	99.92%	100.00%
Ending time (s)	7,200	3,922
Ending time (min)	120.00	65.37
Ending condition	Time limit	All parked

TABLE IV
OPERATIONAL AND SELECTED ENVIRONMENTAL AGGREGATES

Metric	Scenario A	Scenario B
Avg. search time (s)	371.29	108.72
Explicit cruising (km)	2,692.06	0.00
Total fuel (kg)	421.87	184.58
Total CO ₂ (kg)	1,301.31	569.37
Total NOx (g)	413.17	190.02
Total PMx (g)	46.53	46.08

Relative to the baseline, smart allocation shortens the simulation ending time by 3,278 s, or 45.53%, while also improving the parking rate from 99.92% to 100.00%. Vehicle-level logs show a 70.72% lower average search time, elimination of explicit cruising during the reservation phase, 56.25% lower fuel consumption, 56.25% lower CO₂, 54.01% lower NOx, and 0.97% lower PMx.

VI. CONCLUSION

This paper presented a Smart Parking Allocation system designed to reduce the “cruising for parking” phenomenon. The Python TraCI control loop coordinates SUMO vehicle routing with PostgreSQL-backed scenario logging, real-time reservation state updates, and dashboard-level comparison of two executable SUMO scenarios. In the database results, Scenario A parks 2,498 of 2,500 planned vehicles by the 7,200 s simulation limit, while Scenario B parks all 2,500 vehicles by 3,922 s. Scenario B therefore improves both parking completion and total simulation time, while also reducing average search time by 70.72%, fuel and CO₂ by 56.25%, NOx by 54.01%, and PMx by 0.97%.

REFERENCES

- [1] D. C. Shoup, “Cruising for parking,” *Transport Policy*, vol. 13, no. 6, pp. 479–486, 2006.
- [2] E. Inci, “A review of the economics of parking,” *Economics of Transportation*, vol. 4, no. 1-2, pp. 50–63, 2015.
- [3] T. Lin, H. Rivano, and F. Le Mouel, “A Survey of Smart Parking Solutions,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229–3253, Dec. 2017.
- [4] Y. Geng and C. G. Cassandras, “New smart parking system based on resource allocation and reservations,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1129–1139, 2013.
- [5] R. Arnott and J. Rowse, “Modeling parking,” *Journal of Urban Economics*, vol. 45, no. 1, pp. 97–124, 1999.

- [6] M. Fosgerau and A. de Palma, “The dynamics of urban traffic congestion and the price of parking,” *Journal of Public Economics*, vol. 105, pp. 106–115, 2013.
- [7] G. Pierce and D. Shoup, “Getting the prices right: An evaluation of pricing parking by demand in San Francisco,” *Journal of the American Planning Association*, vol. 79, no. 1, pp. 67–81, 2013.
- [8] A. Wegener, M. Piórkowski, M. Raya, H. Hellbrück, S. Fischer, and J.-P. Hubaux, “TraCI: an interface for coupling road traffic and network simulators,” *Proceedings of the 11th Communications and Networking Simulation Symposium*, 2008, pp. 155–163.
- [9] S. Krauss, “Microscopic modeling of traffic flow: Investigation of collision free vehicle dynamics,” Ph.D. dissertation, DLR, Cologne, Germany, 1998.
- [10] M. Behrisch, L. Bieker, J. Erdmann, and D. Krajzewicz, “SUMO Simulation of Urban Mobility: An Overview,” *Proceedings of SIMUL 2011, The Third International Conference on Advances in System Simulation*, 2011.